Group 57 Final Report:
Image-Based Assistive Clothing Classification Using Machine Learning

Khushii Saini, Nika Khajehpour, Himasha Karunathilake
{sainik19,khajehn,karunath}@mcmaster.ca

Dataset: Fashion-MNIST (Kaggle)

1 Introduction

The motivation for this project is to develop a machine learning model that can automatically classify clothing items from images. With text-to-speech integration, this project can be applied in assistive technology to help visually impaired users recognize and differentiate clothing items more independently. This model is structured as a supervised multiclass image classification problem, and each image corresponds to one of the following ten clothing categories:

Clothing Category	Label
T-shirt/Top	0
Trouser	1
Pullover	2
Dress	3
Coat	4
Sandal	5
Shirt	6
Sneaker	7
Bag	8
Ankle Boot	9

Table 1: Fashion-MNIST clothing categories and their corresponding label indices.

To approach this, we first implemented a Logistic Regression model as a baseline and then developed a Convolutional Neural Network (CNN) designed to capture visual features within the images. In our report, we outline the implementation process, evaluation results, and comparison between the two models to determine which approach performs best for the classification task.

2 Related Work

To determine which approach would work best for our project, we reviewed several studies, many of which emphasized that convolutional neural networks (CNNs) are one of the most effective methods for image classification. Earlier work showed that CNNs outperform traditional models by learning spatial features directly from image data (Lao and Jagadeesh, 2015; Hariharan and Prasath, 2019). Along with CNNs, Ayongo et al. (2024) suggested using dropout regularization to address overfitting, so we decided to include both dropout and L1/L2 regularization in our model. Similarly, previous research has shown that techniques such as ReLU activations and parameter tuning can improve model performance (Krizhevsky et al., 2012; Sharma and Phonsa, 2021). We applied ReLU activations throughout our CNN architecture, and we plan to further refine parameters like batch size and epochs in our next milestone.

Since our last project report, we made several updates to improve both the reliability and accessibility of our model, and we found strong support for these choices in recent work. One of the first updates we made was implementing fixed random seeds to reduce training variance. Controlling non-determinism in machine learning workflows is important because reproducibility is essential for avoiding unexpected variation during testing (Dutta et al., 2022). We also added text-to-speech functionality to better support the assistive goal of our project. This decision is supported by recent research on a system called "iSight", which demonstrates that combining computer vision with audio feedback can meaningfully assist visually impaired users in identifying and managing their clothing (Rocha et al., 2025).

3 Dataset

Our project uses the Fashion-MNIST dataset, developed by Zalando Research, which is a dataset for image-based clothing classification. The dataset consists of 70,000 grayscale images of clothing

items and accessories, each with a resolution of 28×28 pixels. The data is divided into 60,000 training images and 10,000 test images, each corresponding to one of the 10 categories mentioned earlier: T-shirt/Top (0), Trouser (1), Pullover (2), Dress (3), Coat (4), Sandal (5), Shirt (6), Sneaker (7), Bag (8), and Ankle Boot (9).

Each CSV file includes a label column (that indicates the clothing type) and 784 feature columns representing the pixel intensity values (0–255). The following preprocessing operations were performed to prepare the data for model training:

- **Feature and Label Separation:** We separated the label column (target variable) from the remaining 784 pixel features.
- **Normalization:** Pixel values were scaled to a 0–1 range by dividing all intensities by 255, allowing faster and more stable convergence during model training since neural networks perform better when features share a similar numerical scale.
- **Train/Validation Split:** The original 60,000 training images were split into 90% training and 10% validation sets (with 10,000 test images already set aside).
- **Reshaping for CNN Input:** Each 784-value vector was reshaped into (28, 28, 1) to restore the original 2D image structure and include the channel dimension required by Keras CNN layers.
- **Label Encoding:** We used one-hot encoding on the target labels, along with the Keras `to_categorical` function, to convert each class into binary vectors and ensure compatibility with the CNN's softmax output layer.

No additional annotation was required since the Fashion-MNIST dataset is pre-labeled.

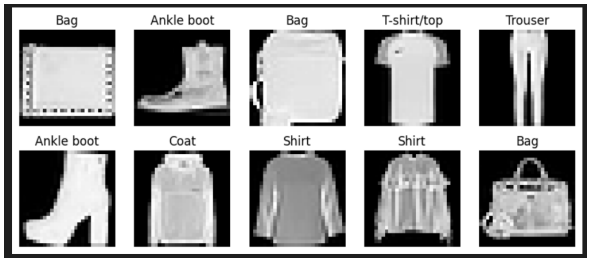

Figure 1: Sample images from the Fashion-MNIST training dataset showing different clothing categories. Each image is 28×28 pixels in grayscale and labeled according to its corresponding clothing class.

4 Features and Inputs

The inputs to our models are 28×28 grayscale images from the Fashion-MNIST dataset. For the Logistic Regression model, each image was flattened into a 784-dimensional vector, while for the CNN, images were reshaped to (28, 28, 1) to preserve spatial information. The pixel values were normalized to a 0–1 range. L1 and L2 regularization were explored as part of our model design to control overfitting, but our final implementation showed that the baseline CNN without explicit regularization achieved the highest validation accuracy. No additional feature engineering, selection, or data augmentation was necessary.
In our CNN, the model automatically learns visual representations through convolutional and pooling layers, allowing it to extract hierarchical features such as edges, textures, and shapes from the images. These learned representations act as embeddings, capturing spatial patterns that contribute to the classification accuracy. The fully connected layers further transform these learned features into high-level representations before passing them to the softmax output layer for final prediction. In contrast, the Logistic Regression model uses raw pixel values directly as input features and does not learn embeddings or hierarchical structures, making it simpler but less capable of capturing visual complexity compared to the CNN.

5 Implementation

Our final system uses two main model families on the Fashion–MNIST dataset: a simple multinomial logistic regression baseline and a small convolutional neural network (CNN) with several regularization variants. All models were trained on the same preprocessed data. We loaded the Kaggle Fashion–MNIST CSV files into `pandas`, separated the `label` column as the target, and normalized pixel values to the range 0 to 1 by dividing by 255. We then split the training data into train and validation sets using `train_test_split(..., test_size=0.1, random_state=42, stratify=y)`, so that class proportions were preserved.

For models that expected flat inputs (logistic regression), each image stayed as a 784-dimensional vector. For the CNN, we reshaped the data to $(28, 28, 1)$ to match the grayscale image shape and used `to_categorical` to one-hot encode the la-

bels.

Baseline model: multinomial logistic regression In our experiments, we used a simple multinomial logistic regression model as our main baseline. It was quick to train, easy to set up in scikit-learn, and it still learned basic relationships between the pixels and the clothing labels, which gave us a clear starting point to compare against the more complex CNN models. Concretely, we implemented it with scikit-learn's `LogisticRegression` (using a higher `max_iter` and the `lbfgs` solver) and trained it on the normalized 784-dimensional pixel vectors from the training split. We then predicted on both the train and validation sets.
For both splits, we computed:

- accuracy,
- macro-averaged precision,
- macro-averaged recall,
- macro-averaged F1,

using `accuracy_score`, `precision_score`, `recall_score`, and `f1_score`. We stored these results in a small summary table (`log_reg_results`) and visualized performance using confusion matrices for the train and validation sets. This logistic regression baseline gave us a simple linear decision boundary in pixel space. It performed much better than a trivial majority-class predictor would, but it still could not properly capture spatial structure in the images. This was exactly what we tried to improve with the CNN.

Logistic Regression Performance Summary:

	Dataset	Accuracy	Precision	Recall	F1-Score
0	Train	0.8810	0.8801	0.8810	0.8804
1	Validation	0.8507	0.8490	0.8507	0.8495

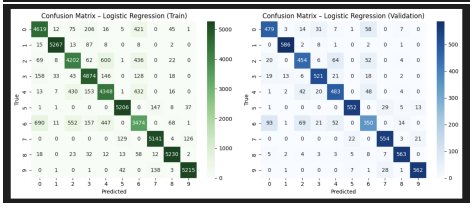

Figure 2: Logistic regression baseline performance. The table reports accuracy, macro precision, macro recall, and macro F1 on the train and validation splits, and the confusion matrices show that the baseline already classifies most classes correctly but still makes systematic mistakes that the CNN aims to reduce.

Main model: CNN and regularization variants Our main model family was a small convolutional neural network built in Keras. We designed the architecture so that we could toggle regularization options while keeping the core structure the same. The shared CNN architecture consisted of:

- an input layer with shape $(28, 28, 1)$ for grayscale images,
- a Conv2D layer with 32 filters and a 3×3 kernel, with ReLU activation,
- a 2×2 max-pooling layer,
- a Conv2D layer with 64 filters and a 3×3 kernel, with ReLU activation,
- a 2×2 max-pooling layer,
- a flatten layer,
- a dense layer with 128 units and ReLU activation (optionally with L1 or L2 kernel regularization),
- a dropout layer with rate 0.3,
- an output dense layer with 10 units and softmax activation for the ten clothing classes.

The loss function was categorical cross-entropy, which matched our one-hot encoded labels, and we optimized it with the Adam optimizer. We trained each CNN variant for 15 epochs with a batch size of 128 and recorded both the training and validation accuracy and loss at each epoch.

Ablations: L1 vs. L2 vs. no regularization To study the effect of regularization and to have multiple versions of our model, we defined a small experiment dictionary with three settings:

- **Baseline (no reg)**: no explicit kernel regularization,
- **L1 (1e-4)**: L1 regularization with weight 1×10^{-4},
- **L2 (1e-4)**: L2 regularization with weight 1×10^{-4}.

For each configuration we:

1. built a model with the chosen regularization type,

2. trained it on `X_train_cnn`, `y_train_cat` and validated on `X_val_cnn`, `y_val_cat`,

3. converted the softmax outputs to integer labels with `argmax` for both train and validation predictions,

4. computed accuracy, macro precision, macro recall, and macro F1 on train and validation,

5. plotted confusion matrices for train and validation,

6. stored the full training history so we could plot training and validation curves for accuracy and loss.

We collected all metrics in a single `results_df` table, which allowed us to directly compare the three CNN variants side by side.

The training curves showed that the "Baseline (no reg)" model reached very high training accuracy but had a larger gap between train and validation, indicating overfitting. L1 regularization was more aggressive and could slightly reduce validation performance. The L2 model with weight 1×10^{-4} initially appeared to provide the best balance between training and validation accuracy in our earlier runs, but after stabilizing experiments with fixed random seeds, the baseline CNN without explicit regularization achieved the highest validation and test accuracy and was selected as our final `best_model` in our experiment loop.

Final model selection and test evaluation During training we tracked the best validation accuracy across all CNN variants, along with the associated predictions on the train and validation sets. Once all three variants were trained, we created a summary table of metrics for the best CNN model and plotted a confusion matrix on the validation set to show how it behaved across classes.
A more detailed discussion of the types of errors this best CNN makes is provided in the Error Analysis section.
Finally, we evaluated `best_model` on the held-out test set `X_test_cnn`. As before, we computed accuracy, macro precision, macro recall and macro F1, and plotted a test-set confusion matrix. In practice, our best CNN variant clearly outperformed the logistic regression baseline on both validation and test metrics, which showed that learning spatial filters with convolutions and adding appropriate regularization gave a meaningful gain over a simpler linear model.

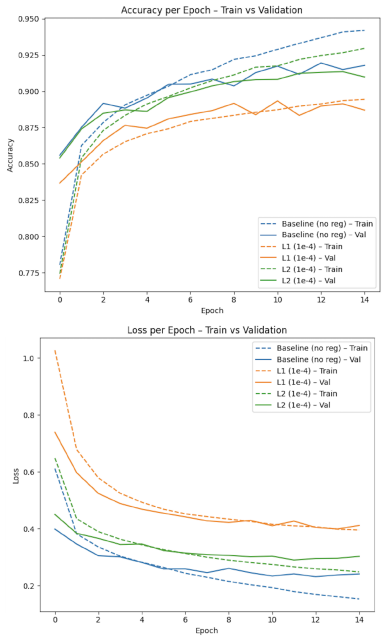

Figure 3: The training curves show that the baseline without regularization reaches the highest training and validation accuracy.

Assistive text to speech integration Since our project was aimed at assistive use, we also wrapped the final CNN in a small text to speech interface using the `pyttsx3` library. The helper function `predict_test_image(index)` took an index from the test set, extracted the corresponding $(28, 28, 1)$ image and true label, and ran `best_model.predict` on that image to obtain the predicted class (via `argmax`). It then mapped the numeric labels to human-readable names using `labels_map` (for example "T-shirt/top" or "Ankle boot"), displayed the image with both the predicted and true class in the title, and called a `speak` function that used `pyttsx3` to say a sentence like "This is a T-shirt/top."
This integration did not change the underlying classifier, but it was important for our application because it turned the model output into something that a visually impaired user could actually hear and interact with in a realistic assistive scenario.

6 Evaluation

For our evaluation strategy, we used a three-way train, validation, and test split rather than cross-validation, as mentioned in earlier sections. Because the dataset contains 70,000 total images, we

determined that a single hold-out validation set was sufficient for reliable model selection and avoided the added computational cost of running k-fold cross-validation.

Data Splitting Strategy: The main training dataset (from the Fashion-MNIST training file) was divided into 90 percent training (54,000 images) and 10 percent validation (6,000 images) using `train_test_split(..., test_size=0.1, random_state=42, stratify=y)`. The `stratify=y` argument ensured that the class distribution remained consistent across both sets, preserving balanced representation for all 10 clothing categories. The test set (10,000 images) was kept entirely separate in a different file and was not used at any point during training or tuning. It served exclusively as a final evaluation set to measure how well our model generalizes to unseen data.

Evaluation Metrics: To evaluate performance, we used accuracy, macro precision, macro recall, and macro F1-score. While accuracy offers a broad sense of the model's performance, the macro-averaged metrics were especially useful because they treat each class equally. This prevented the evaluation from being dominated by high-performing classes such as "Trousers" while downplaying more challenging ones such as "Shirts." We also examined confusion matrices for each model and split to understand specific misclassification patterns.

Adequacy of Metrics: The metrics we proposed in our progress report proved to be appropriate for the final evaluation. Combining numerical results with confusion matrix visualizations helped us interpret the model's strengths and weaknesses more clearly. Although the precision and recall scores were informative, the confusion matrices were particularly useful for identifying semantic overlaps such as the difficulty in separating "Coats" from "Pullovers," which is relevant to improving the user experience for our assistive technology focus.

Model Performance

Baseline (Logistic Regression): The logistic regression model reached a training accuracy of about 88.10 percent and a validation accuracy of about 85.07 percent, confirming the need for a more expressive model architecture.

CNN Variants: We evaluated three CNN versions with different regularization strategies. Although our earlier results suggested that the L2-regularized CNN performed best, our updated experiments, now stabilized with fixed random seeds, showed that the baseline CNN without any regularization actually achieved the highest performance. It reached a validation accuracy of 91.78 percent, which was slightly higher than the L2 model at 90.98 percent and the L1 model at 88.70 percent.

Final Test Results: The best-performing model, the baseline CNN, was then evaluated on the held-out test set and achieved a final test accuracy of 91.89 percent. The close alignment between the validation and test scores suggests that our splitting strategy was effective in preventing overfitting and provided a reliable estimate of the model's real-world performance.

7 Progress

Since the progress report, we focused on refining our existing implementation and incorporating several technical improvements based on both TA feedback and our own observations. The main updates include adding a random seed for reproducibility, enabling verbose output to better monitor training progress, printing misclassified images for interpretability, and integrating a text-to-speech feature using the Python library pyttsx3 to align the project more closely with its assistive technology goal.

We also experimented with data transformations and augmentation, but found that these techniques lowered our accuracy to around 90 percent. This is likely because our model is intentionally lightweight and does not generalize well when stronger augmentations are introduced. After testing, we decided not to include augmentation in order to maintain better performance and avoid unnecessary overfitting.

We had also previously considered implementing cross-fold validation. After reviewing the structure and size of our dataset in more depth, we concluded that it is large enough (70,000 images) for a standard train, validation, and test split. Using cross-validation in this case would add computational cost without providing a meaningful benefit. We also considered early stopping, but since we train

for only 15 epochs, we determined that it would have minimal impact and was not necessary for our setup.

Overall, our plan has stayed consistent with our original proposal. In our earlier report we identified the CNN with L2 regularization as the strongest model, but after stabilizing our experiments with random seeds, we found that the standard CNN (without regularization) actually performs the best, achieving a test accuracy of 91.89 percent. Because of this, we shifted to using the standard CNN as our main model. We continued tuning hyperparameters such as batch size, dropout rate, and learning rate, which improved the stability of our results and supported our broader goal of designing a model suitable for accessibility-focused applications.

8 Error Analysis

To better understand our model's behavior, we examined misclassified validation images and analyzed the validation confusion matrix from the final CNN model. The model performs very well on distinct classes such as sneakers, bags, and ankle boots, which have more recognizable silhouettes and textures. Most errors appear in visually similar categories, for example shirts vs. T-shirts or tops and coats vs. pullovers, where grayscale shading and overall shape make the items difficult to distinguish, even for humans.

We reviewed several misclassified samples alongside their predicted and true labels, and many of these errors occurred on ambiguous or low-contrast images where the outline of the clothing item is hard to see.

From a quantitative standpoint, the confusion matrix shows that classes with greater visual variation such as "Dress" and "Shirt" have slightly lower recall compared to simpler items like "Trouser." Even so, the overall accuracy and F1-scores remain well balanced across categories, which suggests that the model generalizes effectively.

If we were to extend this project further, we would consider slightly expanding the CNN architecture, for example adding another convolutional block, to help capture more detailed spatial features. We would also explore lightweight data augmentation techniques such as small rotations or brightness adjustments, tuned carefully to avoid overfitting.

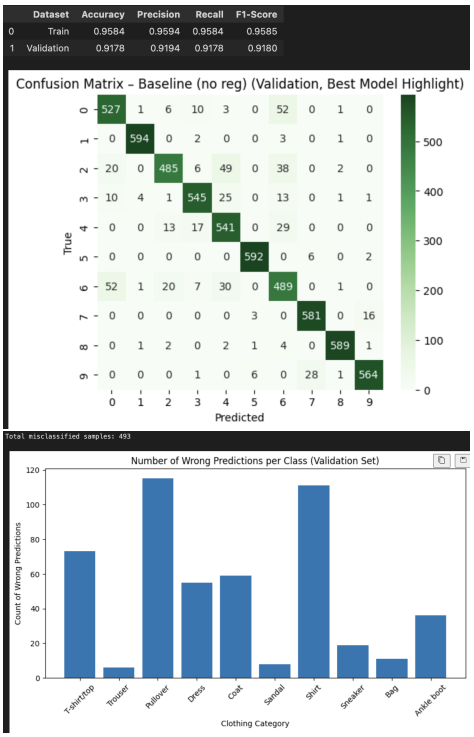

Figure 4: Validation performance and error breakdown for the selected best CNN model (baseline without explicit regularization). The confusion matrix shows that most classes are predicted correctly, while the bar chart counts wrong predictions per true class, revealing that categories like pullover and shirt generated the highest number of errors.

Another promising direction would be testing transfer learning with a pre-trained CNN to strengthen feature extraction for more complex garment types.

Overall, our error analysis and visual inspection indicate that while the model performs well, future work should focus on improving class separation for visually similar clothing items and further adjusting regularization and model depth to enhance robustness.

Team Contributions

Khushii led the implementation of the code and improved our earlier version by adding key features such as fixed random seeds, misclassified image visualization, and the text-to-speech component. Nika and Himasha reviewed the updated code, analyzed the results, and reported on the findings. Nika and Himasha also contributed to writing and refining the LaTeX documentation for the report.

References

Shuwa Ayongo, Muhammad Abdullahi, Mustapha A. Bagiwa, and Alice O. Matemilolas. 2024. Clothing regularized image classification multiple model convolutional using neural networks (rmcnn). In *Science World Journal Vol. 19(No 2) 2024*.

Saikat Dutta, Anshul Arunachalam, and Sasa Misailovic. 2022. To seed or not to seed? an empirical analysis of usage of seeds for testing in machine learning projects. *2022 IEEE Conference on Software Testing, Validation and Verification (ICST)*.

Shanmugasundaram Hariharan and Ram Prasath. 2019. Deep learning approaches for fashion image classification. *International Journal of Pure and Applied Mathematics*.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*.

Brian Lao and Karthik Jagadeesh. 2015. Convolutional neural networks for fashion classification and object detection. *Standford University*.

Daniel Rocha, Celina Leão, Filomena Soares, and Vítor Carvalho. 2025. isight: A smart clothing management system to empower blind and visually impaired individuals. *AI-Based Image Processing and Computer Vision*.

Atul Sharma and Gurbakash Phonsa. 2021. On the impact of relu and hyperparameters in cnn training. *Journal of Machine Learning Research*.